

Informe técnico: Implementación y comparación de algoritmos ráster

Proyecto: OperacionDDA

2 de junio de 2026

Índice

1. Introducción	2
2. Algoritmos implementados	2
2.1. Líneas	2
2.2. Circunferencias	2
2.3. Relleno	2
3. Metodología de pruebas	3
4. Capturas de pantalla de las pruebas	3
5. Explicación paso a paso	6
6. Comparación entre algoritmos implementados	7
7. Conclusiones	7
8. Apéndice: cómo generar las capturas y completar la tabla	8

1. Introducción

Este documento resume la implementación, pruebas y comparaciones de los algoritmos ráster desarrollados en el proyecto "OperacionDDA". El objetivo es explicar el funcionamiento interno, mostrar resultados de pruebas con capturas de pantalla y extraer conclusiones sobre comportamiento y eficiencia.

2. Algoritmos implementados

Se implementaron algoritmos para tres temas principales: líneas, circunferencias y relleno. A continuación se describen.

2.1. Líneas

- **DDA (Digital Differential Analyzer)**: algoritmo de interpolación que avanza en el eje con mayor diferencia y calcula incrementos fraccionales para el otro eje; se redondea a la posición de píxel más cercana. Implementación registra cada punto trazado.
- **Bresenham**: algoritmo entérico que usa decisión por error acumulado para seleccionar el próximo píxel sin operaciones de coma flotante (solo sumas y comparaciones). Ideal para rendimiento en entornos ráster.
- **Punto medio (Midpoint line)**: versión basada en la evaluación de una función de decisión similar a Bresenham, que compara posiciones relativas para elegir el siguiente píxel.

Para cada algoritmo de líneas se provee: entrada de coordenadas $(x1,y1)-(x2,y2)$, visualización sobre un plano centrado, lista de puntos generados y contador de pasos.

2.2. Circunferencias

- **Punto medio (Midpoint circle)**: calcula octantes usando la decisión basada en la función p y refleja puntos en las 8 simetrías; registra lista de puntos y píxeles estimados.
- **Bresenham para círculos**: variante entérica optimizada del algoritmo de círculo que mantiene una variable de decisión y actualiza coordenadas sin trigonometría costosa.
- **Paramétrico**: usa $x = xc + r \cos(\theta)$, $y = yc + r \sin(\theta)$ con muestreo en pasos sobre θ , útil como referencia aunque requiere trigonometría y redondeo.

2.3. Relleno

- **Flood Fill (stack-based)**: algoritmo de llenado por pila (DFS) que pinta píxeles adyacentes si su color coincide con el objetivo.
- **Boundary Fill**: similar al anterior pero se detiene ante un color de borde especificado.

- **Scanline Flood Fill:** mejora acelerando el relleno por tramos horizontales (scanlines) en lugar de píxel a píxel, menos operaciones en áreas grandes.

3. Metodología de pruebas

Pruebas realizadas en el formulario principal y los formularios dedicados a circunferencias y relleno. Se usaron:

- Diferentes pares de puntos para líneas (pendientes positivas, negativas, verticales y horizontales).
- Radios variados para circunferencias (pequeños y grandes) y comparación entre algoritmos.
- Formas cerradas dibujadas manualmente para probar los algoritmos de relleno. En el modo "Paso a paso" se activó la generación progresiva de píxeles y se tomaron capturas.

4. Capturas de pantalla de las pruebas

Coloque en el directorio `screenshots/` las imágenes generadas ejecutando la aplicación y usando el botón de captura de su entorno (o `PrtScr`). A continuación se incluyen las referencias a las capturas utilizadas en el informe. Reemplace los archivos con las capturas reales.

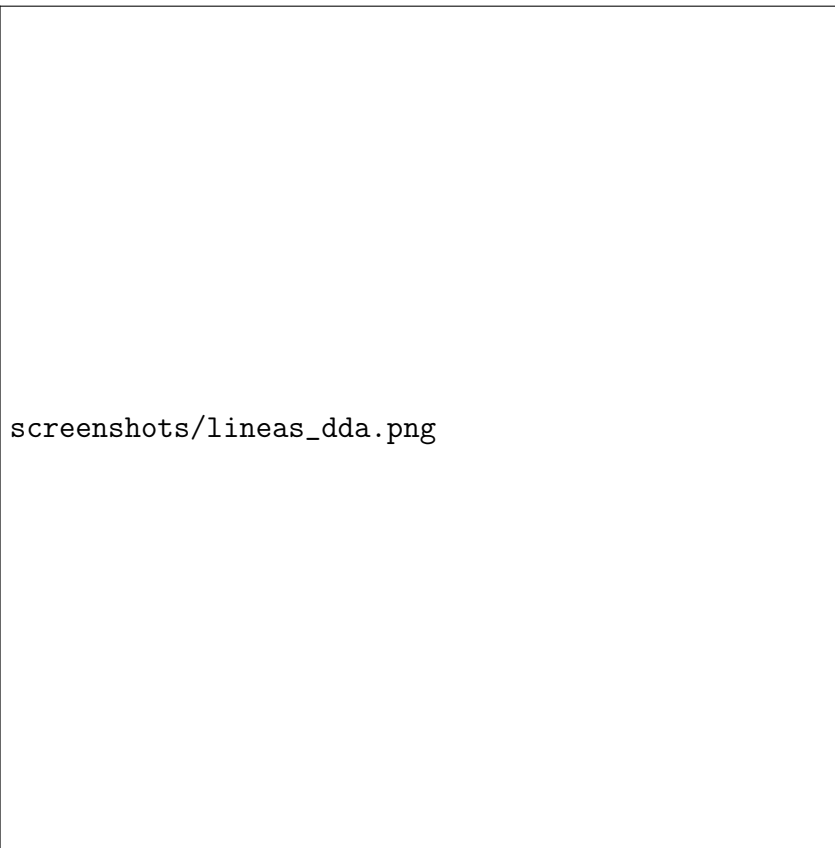


Figura 1: Trazado con DDA (ejemplo)



screenshots/lineas_bresenham.png

Figura 2: Trazado con Bresenham (ejemplo)



screenshots/circunferencia_parametrico.png

Figura 3: Circunferencia - método paramétrico



screenshots/relleno_scanline.png

Figura 4: Relleno con Scanline Flood Fill en ejecución

5. Explicación paso a paso

El proyecto incluye un modo "Paso a paso" que permite observar cómo el algoritmo genera píxeles uno a uno o por bloques:

1. Al activar "Paso a paso," el controlador genera previamente la lista ordenada de coordenadas (sin pintarlas en pantalla).
2. Un temporizador avanza un contador `currentStep` y el pintado solo dibuja los primeros `currentStep` puntos de la lista. De esta forma se puede ver la progresión.
3. La lista de puntos se muestra en la lista GUI (ListBox) y el contador de pasos se actualiza en cada tick.
4. Para los algoritmos de relleno el motor procesa un número limitado de píxeles por tick (parámetro "píxeles por paso") para mantener la UI responsiva.

Ejemplo de explicación (Bresenham — líneas):

- Inicializar variables: $dx, dy, sx, sy, err = dx - dy$.
- Añadir primer punto (x, y) a la lista.
- En cada iteración evaluar $e2 = 2 \cdot err$. Si $e2 > -dy$ actualizar $err- = dy$ y $x+ = sx$. Si $e2 < dx$ actualizar $err+ = dx$ y $y+ = sy$.

- Registrar el nuevo punto en la lista.
- Visualizar progresivamente según `currentStep`.

6. Comparación entre algoritmos implementados

A continuación se proponen criterios de comparación y una tabla que puede completarse con los resultados experimentales:

- **Precisión:** proximidad a la curva teórica (paramétrico tiende a mayor suavidad, Bresenham y Punto medio son exactos en píxeles discretos).
- **Costo computacional:** operaciones por píxel (Bresenham y Punto medio usan solo sumas/enteros; paramétrico usa trigonometría y redondeo).
- **Número de pasos/píxeles:** medido por los contadores internos tras la ejecución.
- **Velocidad práctica:** tiempo observado en la UI; modo paso a paso permite contar y comparar.

Algoritmo	Operaciones clave	Precisión	Pixels generados	Comentarios
DDA	coma flotante, redondeo	buena	(rellenar)	más simple, menos
Bresenham	enteros, suma	alta	(rellenar)	eficiente y rápido
Punto medio	enteros, decisión	alta	(rellenar)	parecido a Bresenham
Paramétrico	trigonometría	variable	(rellenar)	referencia, más costoso
Flood Fill	comparación de color	exacto	(rellenar)	sensible al target
Boundary Fill	comparación borde	exacto	(rellenar)	requiere borde bien definido
Scanline	tramos horizontales	muy eficiente	(rellenar)	menos operaciones en áreas grandes

Cuadro 1: Comparación cualitativa

Recomiendo ejecutar pruebas automáticas y completar la columna "Pixels generados" con los valores devueltos por los métodos `getPuntos()/getPíxeles()` en las clases correspondientes.

7. Conclusiones

- Para líneas en raster, Bresenham y Punto medio son preferibles por su rendimiento entero y alta precisión en píxeles.
- DDA es didáctico y sencillo de implementar, útil para entender interpolación, pero menos eficiente.
- Para circunferencias, el algoritmo de punto medio y Bresenham son adecuados; el método paramétrico sirve como referencia visual pero es más costoso.
- En relleno, Scanline ofrece ventajas en rendimiento en áreas amplias; Flood Fill y Boundary Fill son útiles para casos simples y demostrativos.
- El modo paso a paso es muy útil para enseñanza: permite ver las decisiones internas y comparar visualmente la evolución entre algoritmos.

8. Apéndice: cómo generar las capturas y completar la tabla

1. Abrir la solución en Visual Studio y compilar.
2. Ejecutar la aplicación desde Visual Studio (Debug/Release según prefiera).
3. En cada formulario (líneas, circunferencias, relleno) introducir parámetros de prueba.
4. Activar "Paso a paso" para capturar la evolución o ejecutar completo para capturar la imagen final.
5. Guardar capturas en la carpeta **screenshots/** y actualizar los nombres en este documento.
6. Ejecutar varias pruebas y anotar los valores de los contadores que aparecen en la UI (Pasos/Píxeles) para completar la tabla comparativa.